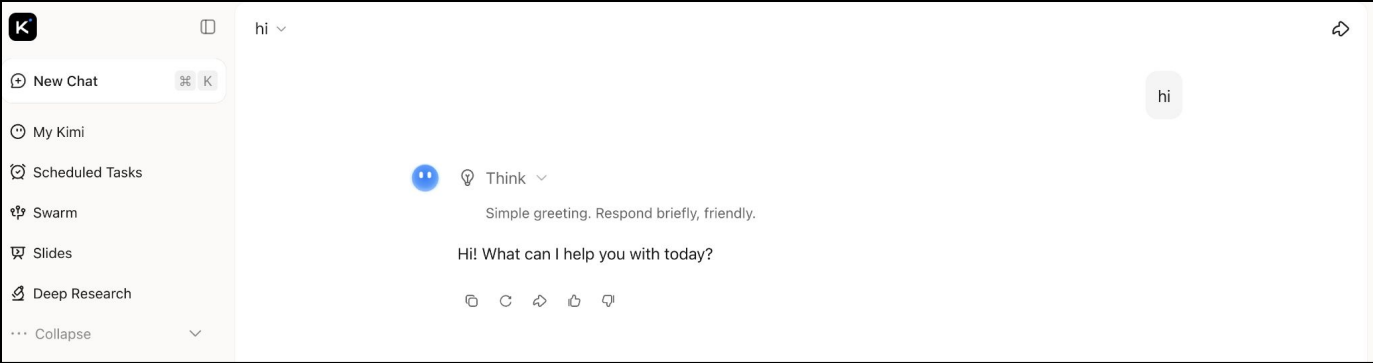
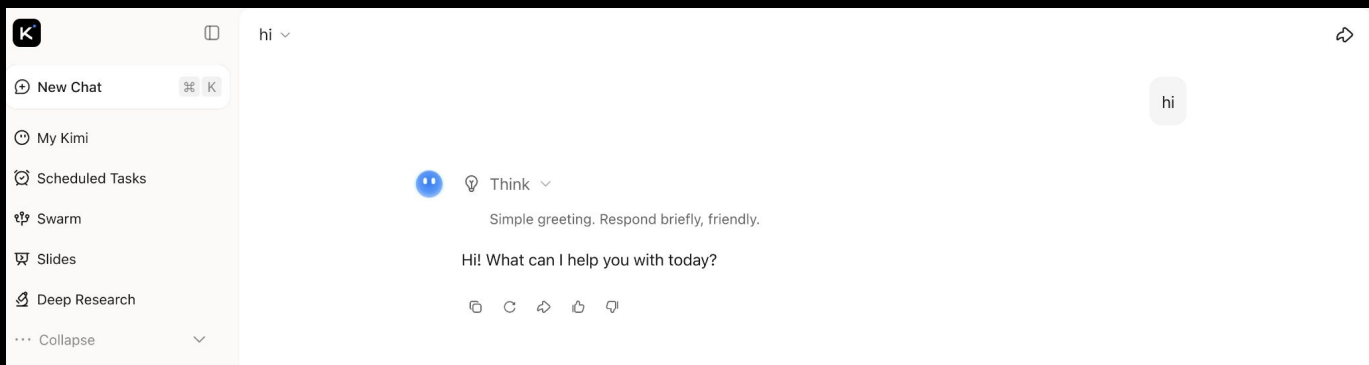


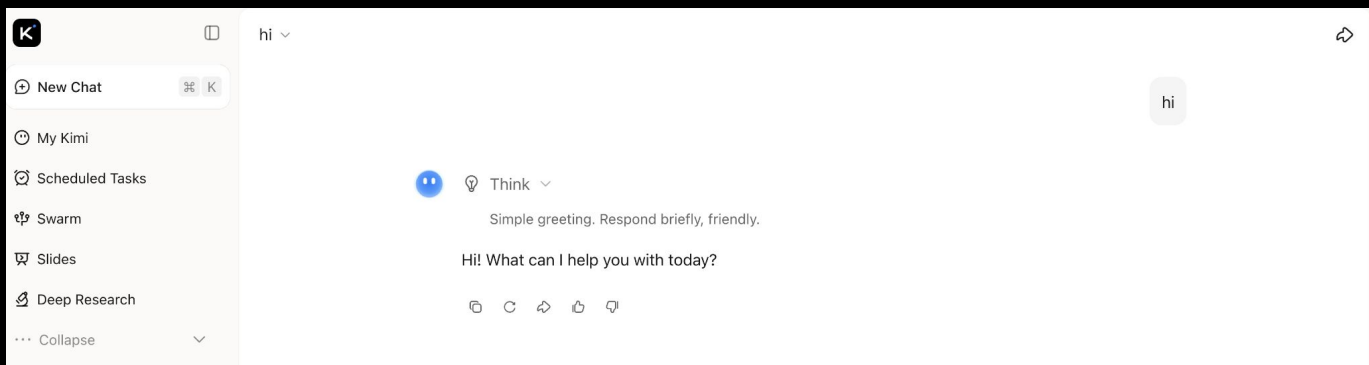
AI kernel programming 101

with tinygrad





Kimi's reply took ~570,000,000,000 multiply-adds



Kimi's reply took ~570,000,000,000 multiply-adds
At 55 tok/s, that's $\approx$ 1.7 trillion multiply-adds every second

**AI is simple math,
repeated at scale**

```
for i in range(n):  
    x += a[i] * b[i]
```

**AI is simple math,
repeated at scale**

**With a Python loop, Kimi's
reply would take ~8 hours**

```
for i in range(n):  
    x += a[i] * b[i]
```

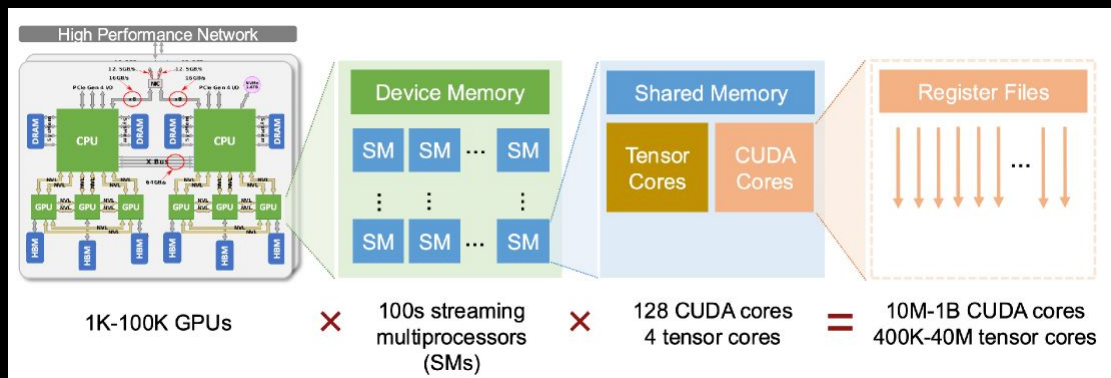
AI is simple math,
repeated at scale

With a Python loop, Kimi's
reply would take ~8 hours

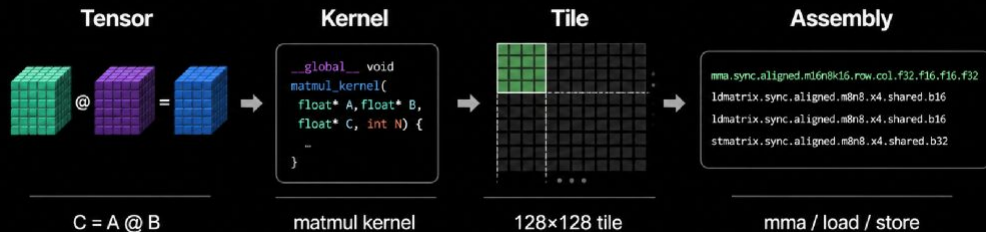
```
for i in range(n):  
    x += a[i] * b[i]
```

We make AI faster by doing many
operations at the same time

AI hardware is Massively Parallel



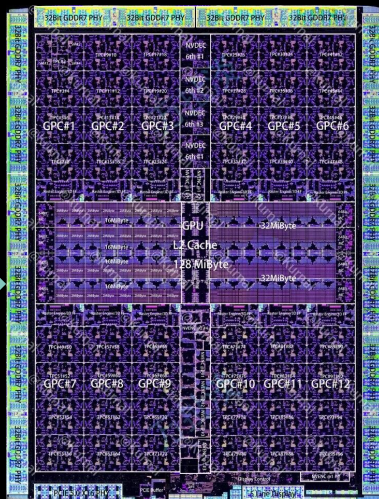
AI software has to find the optimal parallelism



AI hardware is Massively Parallel



Our main training/compute machines. Notice the 400Gbps switch in the center.



1. Preliminary performance estimates for H100 based on current expectations and subject to change in the shipping product
2. Effective TFLOPS / TOPS using the Sparsity feature

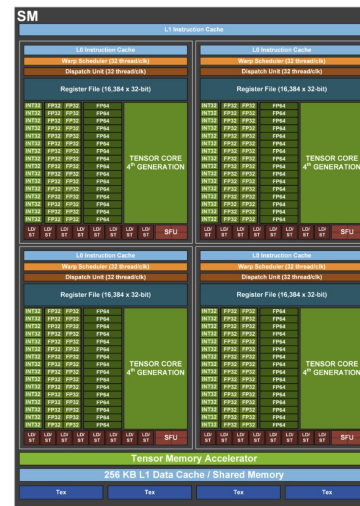
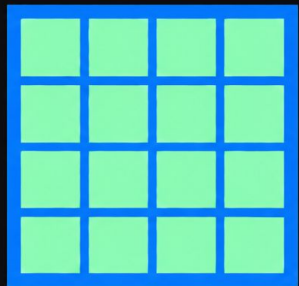


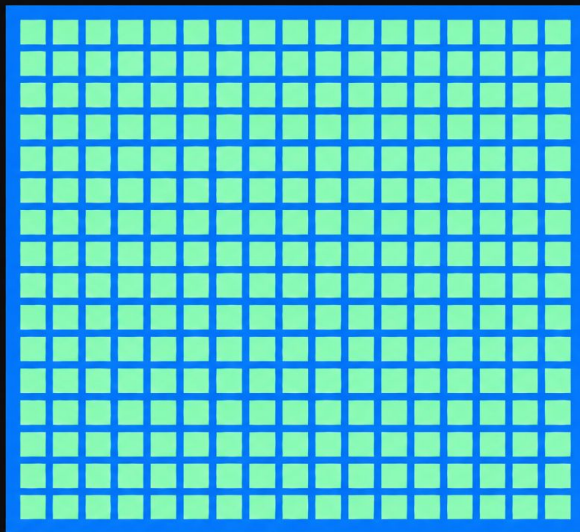
Figure 4. GH100 streaming multiprocessor

This workshop will run on a CPU

Same ideas apply to a GPU at a larger scale



CPU



GPU

**GPUs have many
workers, software has
to keep them busy**



tinygrad is a complete stack for GPU programming in pure Python

tinygrad	:	1740	in	6	files
tinygrad/codegen	:	2053	in	15	files
tinygrad/engine	:	538	in	3	files
tinygrad/llm	:	1498	in	6	files
tinygrad/mixin	:	1672	in	8	files
tinygrad/nn	:	1550	in	6	files
tinygrad/renderer	:	3552	in	13	files
tinygrad/runtime	:	8046	in	41	files
tinygrad/schedule	:	1127	in	7	files
tinygrad/uop	:	2492	in	10	files
tinygrad/viz	:	2020	in	4	files

ops: 78
flags: 55
core lines: 9622
total lines: 26288
26K lines

tinygrad is a complete stack for GPU programming in pure Python

```
tinygrad           : 1740 in 6 files
tinygrad/codegen   : 2053 in 15 files
tinygrad/engine    : 538 in 3 files
tinygrad/llm       : 1498 in 6 files
tinygrad/mixin     : 1672 in 8 files
tinygrad/nn        : 1550 in 6 files
tinygrad/renderer  : 3552 in 13 files
tinygrad/runtime   : 8046 in 41 files
tinygrad/schedule  : 1127 in 7 files
tinygrad/uop       : 2492 in 10 files
tinygrad/viz       : 2020 in 4 files
```

```
ops: 78
flags: 55
core lines: 9622
total lines: 26288
26K lines
```

```
9     license = 'MIT'
10    readme = "README.md"
11    requires-python = ">=3.11"
12    dependencies = []
```

0 Python dependencies

tinygrad is a complete stack for GPU programming in pure Python

```
tinygrad          : 1740 in 6 files
tinygrad/codegen  : 2053 in 15 files
tinygrad/engine   : 538 in 3 files
tinygrad/llm      : 1498 in 6 files
tinygrad/mixin    : 1672 in 8 files
tinygrad/nn       : 1550 in 6 files
tinygrad/renderer : 3552 in 13 files
tinygrad/runtime  : 8046 in 41 files
tinygrad/schedule : 1127 in 7 files
tinygrad/uop      : 2492 in 10 files
tinygrad/viz      : 2020 in 4 files
```

```
ops: 78
flags: 55
core lines: 9622
total lines: 26288
26K lines
```

```
9     license = 'MIT'
10    readme = "README.md"
11    requires-python = ">=3.11"
12    dependencies = []
```

0 Python dependencies

Accelerators

tinygrad already supports numerous accelerators, including:

- ☒ [OpenCL](#)
- ☒ [CPU](#)
- ☒ [METAL](#)
- ☒ [CUDA](#)
- ☒ [AMD](#)
- ☒ [NV](#)
- ☒ [QCOM](#)
- ☒ [WEBGPU](#)

runs on a wide range of hardware (CPU, Apple, AMD, NVIDIA)

tinygrad's abstractions

**Tensor is a high level
numpy style API**

```
1 from tinygrad import Tensor
2
3 out = Tensor([1]) + Tensor([2])
4 out.realize()
5
```

Recap:

**AI is simple math,
repeated at scale**

**With a Python loop, Kimi's
reply would take ~8 hours**

```
for i in range(n):  
    x += a[i] * b[i]
```

**We make AI faster by doing many
operations at the same time**

Workshop warm up:

`x += a * b` at scale

 0.py > ...

```
1  # this is a warmup
2
3  def solve(a, b):
4      out = 0.0
5      for i in range(len(a)):
6          out += a[i] * b[i]
7      return out
8
```

`python workshop.py submit 0.py --name <NAME>`

Workshop warm up:

$x \ += \ a \ * \ b$ at scale

```
0.py > ...  
1   from tinygrad import Tensor  
2  
3   def solve(a, b):  
4       |   return (Tensor(a) * Tensor(b)).sum()  
5  
6  
7  
8
```

python workshop.py submit 0.py --name <NAME>

VIZ=1



Profiler

Callify 3 Buffers n1

Schedule 4 Kernels n1

▾ schedule to linear - 1 (2)

▾ prepare_rangeify (2)

multi_pm

earliest rewrites - 5

▸ get_kernel_graph (9)

▸ resolve linear call - 1 (1)

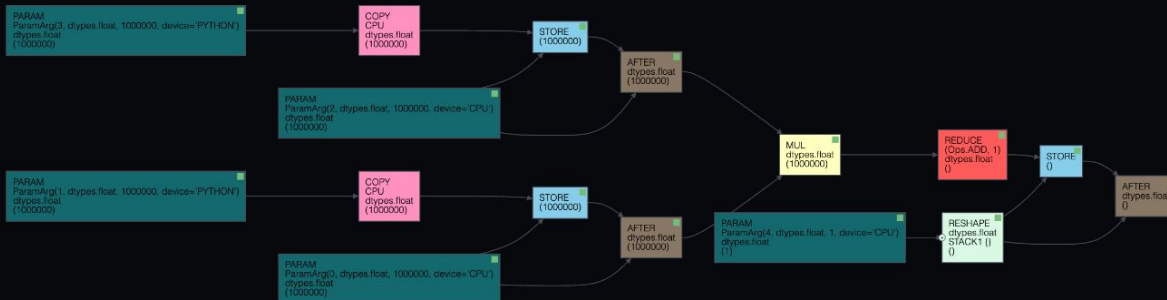
▸ create COPY kernels for SC
memory plan - 1

r_250_1000_4

r_250

HCQ Compile 4 Kernels n1

HCQ Link 4 Kernels n1



☒ Show indexing (r)

☐ Show all CALL src (c)

☐ Show SINK (s)

[prepare.py:222](#)

```
tsink = graph_rewrite(tsink,  
pm_mops+earliest_rewrites,  
bottom_up=True,  
name="earliest rewrites")
```

```
c0 = UOp(Ops.PARAM, (), ParamArg(3, dtype:float, 1000000, device='PYTHON'))  
c1 = UOp(Ops.PARAM, (), ParamArg(2, dtype:float, 1000000, device='CPU'))  
c2 = c1.copy_to_device('CPU')  
c4 = c0.after(c0.store(c2))  
c5 = UOp(Ops.PARAM, (), ParamArg(1, dtype:float, 1000000, device='PYTHON'))  
c6 = UOp(Ops.PARAM, (), ParamArg(0, dtype:float, 1000000, device='CPU'))  
c7 = c6.copy_to_device('CPU')  
c9 = c5.after(c5.store(c7))  
c10 = UOp(Ops.PARAM, (), ParamArg(4, dtype:float, 1, device='CPU'))  
c11 = UOp(Ops.STACK, (), STACK1 [1])  
c12 = UOp(Ops.RESHAPE, (c10, c11))  
c13 = c4*c9  
c14 = c13._rop(Ops.ADD, 0)  
c16 = c12.after(c12.store(c14))
```

0 1 2 3 4 5

C output

```
void r_250(float* restrict data0_1, float* restrict data1_250) {  
    float buf0[1];  
    *(buf0+0) = 0.0f;  
    for (int Ridx0 = 0; Ridx0 < 250; Ridx0++) {  
        float val0 = (*(data1_250+Ridx0));  
        *(buf0+0) = (*(buf0+0))+val0;  
    }  
    *(data0_1+0) = *(buf0+0);  
}
```

LLVMIR output (DEV=CPU:LLVM)

```
define void @r_250(float* noalias align 32 %data0, float* noalias align 32 %data1) #0
{
    %reg_0 = alloca [1 x float]
    %v0 = getelementptr inbounds float, float* %reg_0, i32 0
    store float 0.0, float* %v0
    br label %loop_entry_0
loop_entry_0:
    br label %loop_latch_0
loop_latch_0:
    %v2 = phi i32 [ 0, %loop_entry_0 ], [ %v2phi, %loop_footer_0 ]
    %v2phi = add i32 %v2, 1
    %v2cmp = icmp ult i32 %v2, 250
    br i1 %v2cmp, label %loop_body_0, label %loop_exit_0
loop_body_0:
    %v3 = getelementptr inbounds float, float* %data1, i32 %v2
    %v4 = load float, float* %v3
    %v5 = getelementptr inbounds float, float* %reg_0, i32 0
    %v6 = load float, float* %v5
    %v7 = fadd nsz arcp contract afn float %v6, %v4
    store float %v7, float* %v5
    br label %loop_footer_0
loop_footer_0:
    br label %loop_latch_0
loop_exit_0:
    %v10 = getelementptr inbounds float, float* %reg_0, i32 0
    %v11 = load float, float* %v10
    %v12 = getelementptr inbounds float, float* %data0, i32 0
    store float %v11, float* %v12
    ret void
}
attributes #0 = { alwaysinline nounwind "no-builtins" "no-trapping-math"="true" }
```

Challenge 1: Faster way to compute this double matmul

```
3 M, K, R, N = 64, 1024, 16, 1024
4 x = Tensor.randn(M, K)
5 a = Tensor.randn(K, R)
6 b = Tensor.randn(R, N)
7 out = x @ (a @ b)
```

**Agents can easily recognize structure
and change the algorithm**

Agents can easily recognize structure and change the algorithm

This is very hard with compiler rules

Same math, ~40× less matmul work

Multiply through the skinny dimension first

```
def solve(x, a, b):  
-   return x @ (a @ b)  
+   return (x @ a) @ b
```

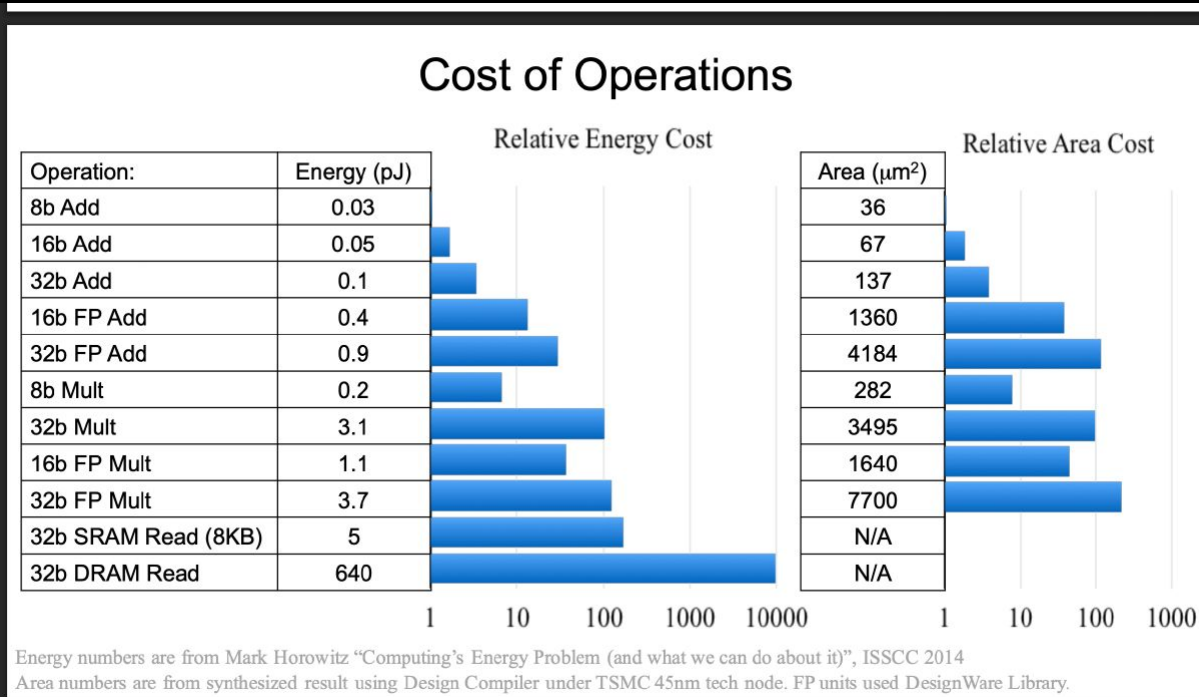
$x @ (a @ b)$ **83.9M MACs**

builds a 1024×1024 intermediate

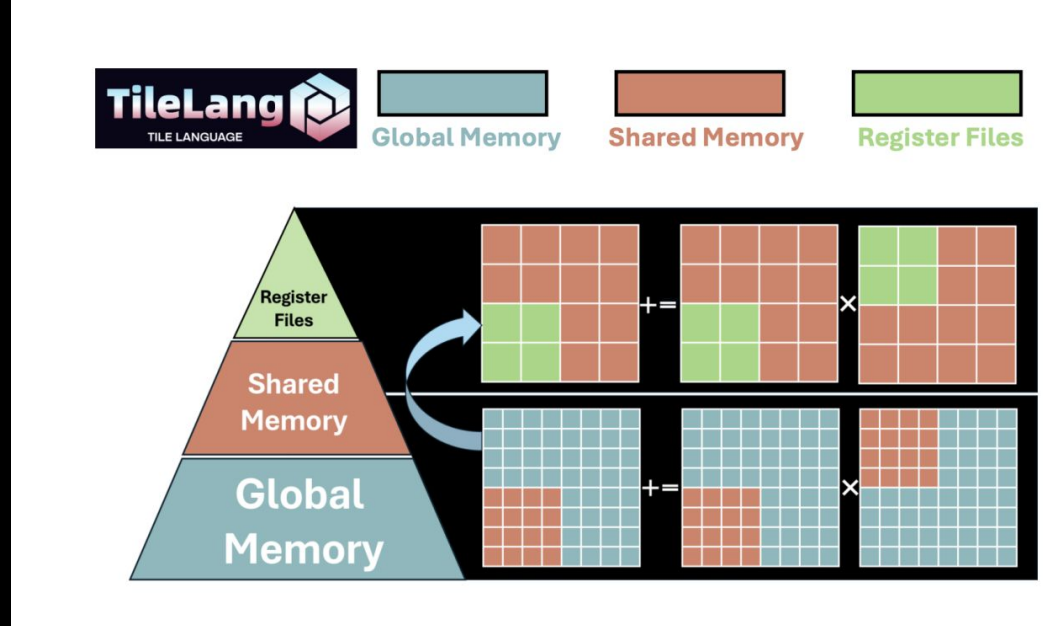
$(x @ a) @ b$ **2.1M MACs**

builds a 64×16 intermediate

Trips to memory are expensive!



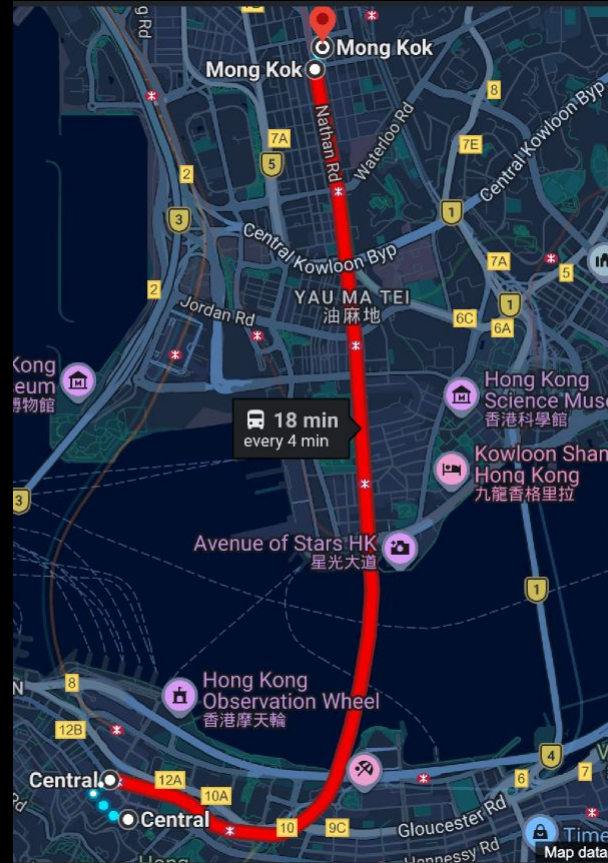
Fast AI kernels use many tricks to minimize going to global memory



Kernel fusion

Kernel fusion

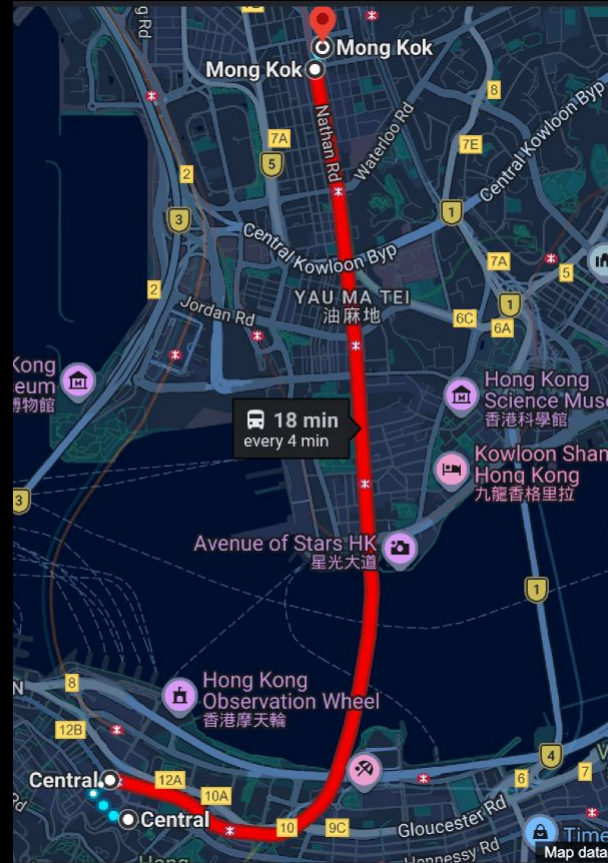
I live in Mong Kok and
have 3 errands in Central



Kernel fusion

I live in Mong Kok and
have 3 errands in Central

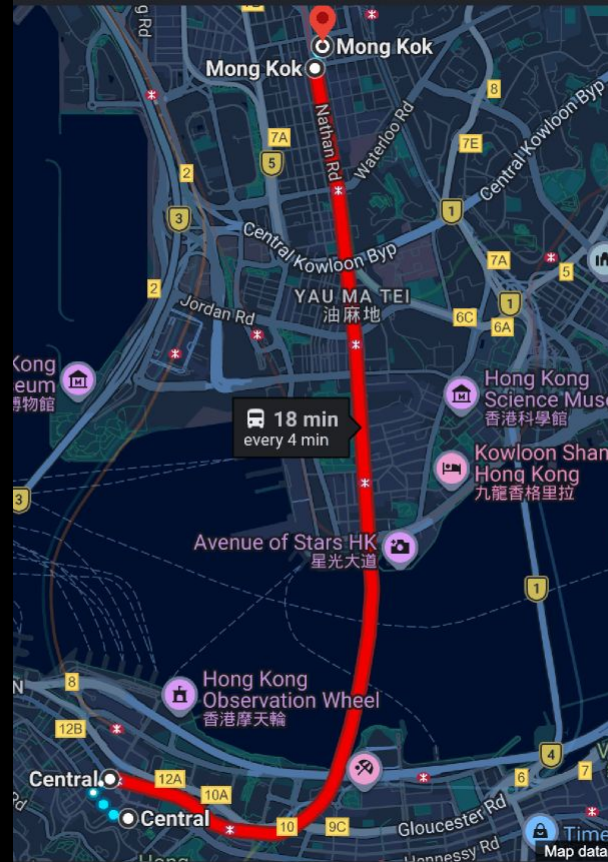
No fusion: travel 3 times



Kernel fusion

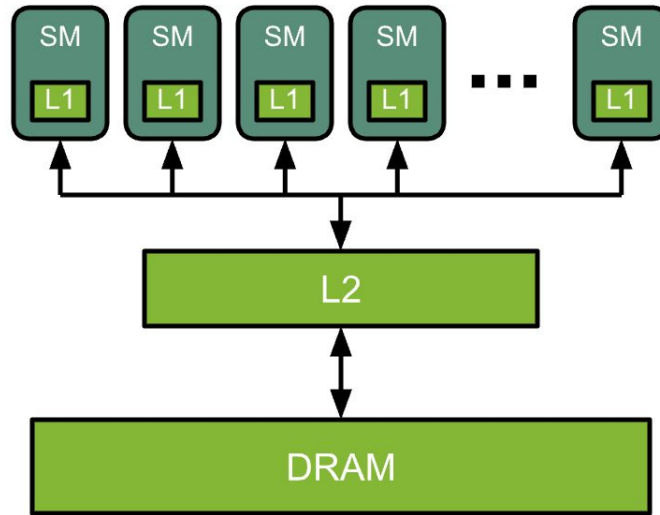
I live in Mong Kok and
have 3 errands in Central

No fusion: travel 3 times
Fusion: travel 1 time



DRAM (memory) is too far from SMs (compute)

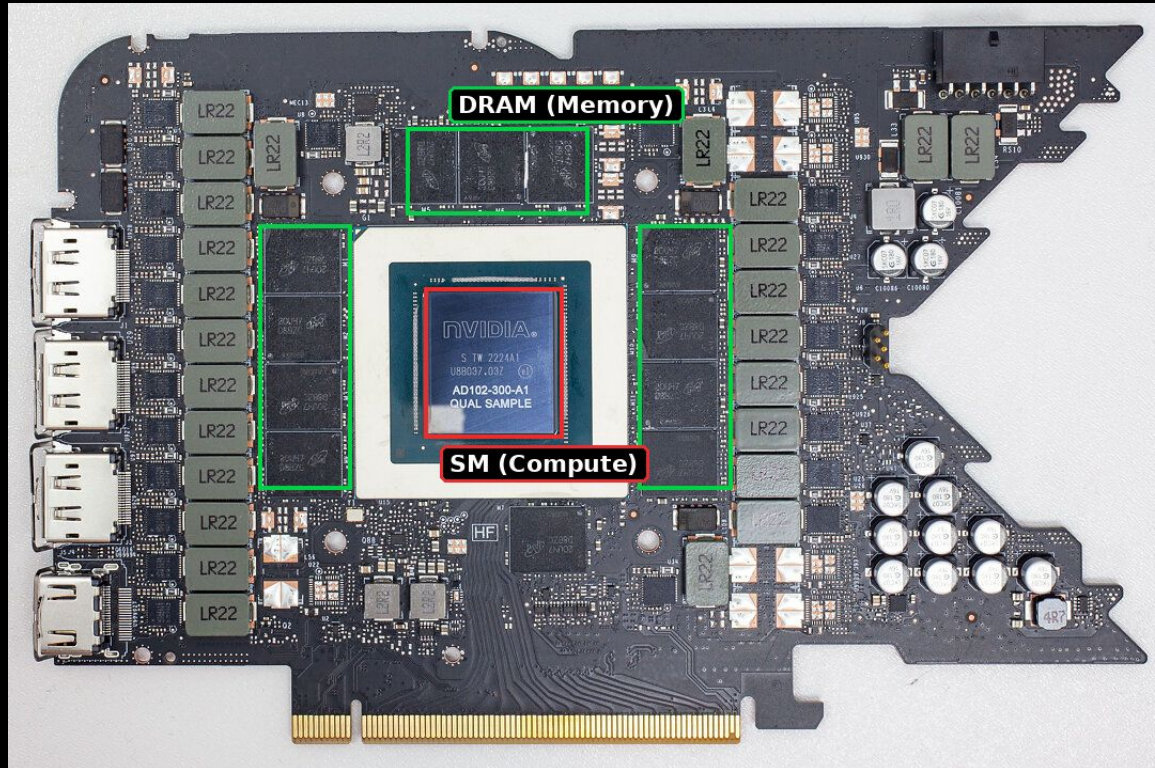
Figure 1. Simplified view of the GPU architecture



DRAM (memory) is too far from SMs (compute)



DRAM (memory) is too far from SMs (compute)



Challenge 2: softmax in one kernel

2.py > ...

```
1  # U0p challenge: implement stable softmax as a custom fused kernel.  
2  # Input/output: float32 [32, 4096].  
3  
4  def solve(x):  
5      |   return x.softmax(-1)  
6
```

VIZ=1 python workshop.py check 2.py

Challenge 2: softmax in one kernel

Softmax is a common operation in LLMs, it turns a list of scores into probabilities.

 Kimi K2: Open Agentic Intelligence

3 / 32 - 239% +     

Taming Muon with QK-Clip To address this issue, we propose a novel weight-cl explicitly constrain attention logits. QK-Clip works by rescaling the query and key pro bound the growth of attention logits.

Let the input representation of a transformer layer be $\mathbf{X}$. For each attention head h , its q are computed as

$$\mathbf{Q}^h = \mathbf{X}\mathbf{W}_q^h, \quad \mathbf{K}^h = \mathbf{X}\mathbf{W}_k^h, \quad \mathbf{V}^h = \mathbf{X}\mathbf{W}_v^h.$$

where $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$ are model parameters. The attention output is:

$$\mathbf{O}^h = \text{softmax}\left(\frac{1}{\sqrt{d}}\mathbf{Q}^h\mathbf{K}^{h\top}\right)\mathbf{V}^h.$$

Challenge 2: softmax in one kernel

Profiler

Callify 1 Buffer n1

View Tensor Graph

add tags - 1

early transform tensor graph

graph_rewrite

replace bufs - 3

View Call

substitute buffers - 1

Schedule 3 Kernels n1

r_32_1024_4

r_32_1024_4

E_32_1024_4

HCQ Compile 3 Kernels n1

HCQ Link 3 Kernels n1



■ Show all CALL src (c)

■ Show SINK (s)

[tensor.py:199](#)

```
if VIZ:
    graph_rewrite(big_sink,
        PatternMatcher([]),
        name="View Tensor Graph")
```

<module> [workshop.py:108](#)

```
if __name__ == "__main__": mai
main workshop.py:92
if args.command == "check": re
local_check workshop.py:65
result = solve(*inputs).realiz
_wrapper tensor.py:771
ret = fn(*args, **kwargs)
inner contextlib.py:85
return func(*args, **kwyds)
realize tensor.py:414
run_linear(*Tensor.linear_with
```

UOp(Ops.SINK, arg=None, src=(
UOp(Ops.MUL, arg=None, src=(
x1=UOp(Ops.FXP2, arg=None

Challenge 2: softmax in one kernel

Profiler

Callify 1 Buffer n1

View Tensor Graph

add tags - 1

early transform tensor graph

graph_rewrite

replace bufs - 3

View Call

substitute buffers - 1

Schedule 3 Kernels n1

r_32_1024_4

r_32_1024_4

E_32_1024_4

HCQ Compile 3 Kernels n1

HCQ Link 3 Kernels n1

find max	subtract max	exp	sum exp	normalize
				

■ Show all CALL src (c)

■ Show SINK (s)

[tensor.py:199](#)

```
if VIZ:
    graph_rewrite(big_sink,
        PatternMatcher([]),
        name="View Tensor Graph")
```

<module> [workshop.py:108](#)

```
if __name__ == "__main__": mai
main workshop.py:92
if args.command == "check": re
local_check workshop.py:65
result = solve(*inputs).realiz
_wrapper tensor.py:771
ret = fn(*args, **kwargs)
inner contextlib.py:85
return func(*args, **kwargs)
realize tensor.py:414
run_linear(*Tensor.linear_with
```

UOp(Ops.SINK, arg=None, src=(
UOp(Ops.MUL, arg=None, src=(
x1=UOp(Ops.EXP2, arg=None

It's taking 3 trips to memory

Schedule 3 Kernels n1

▼ schedule to linear - 1 (2)

▸ prepare_rangeify (3)

▼ get_kernel_graph (9)

▸ run_rangeify (5)

▸ symbolic_reduce_c

limit buffers

View Rangeify

stage to store - 10

▸ split kernels - 3 (3)

remove indexing from

remove views from th

View Kernel Graph

▸ resolve linear call - 1 (1)

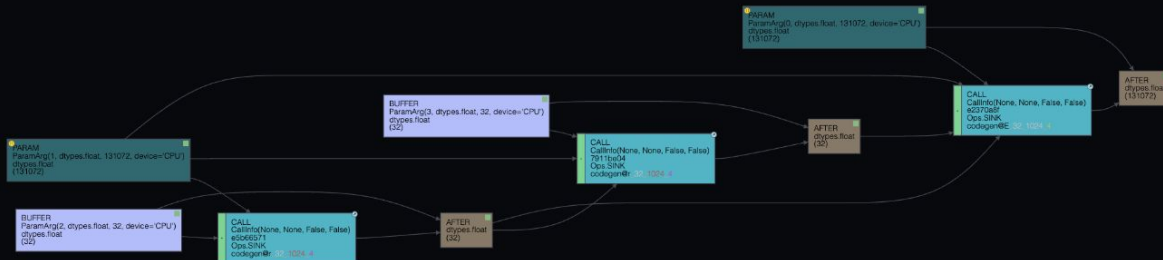
create COPY kernels for ε

memory plan - 2

r_32_1024_4

r_32_1024_4

E_32_1024_4



[rangeify.py:398](#)

```
if VIZ:
    graph_rewrite(tsink,
        PatternMatcher([]),
        name="View Kernel Graph")
```

```
UOp(Ops.SINK, arg=None, src=
    UOp(Ops.AFTER, arg=None,
        x1:=UOp(Ops.PARAM, arg=
            UOp(Ops.CALL, arg=CallI
                UOp(Ops.SINK, arg=Ker
                    UOp(Ops.END, arg=No
                        UOp(Ops.STORE, ar
                            UOp(Ops.INDEX,
                                UOp(Ops.PARAM
                                    x8:=UOp(Ops.A
                                        UOp(Ops.MUL
                                            x10:=UOp(
                                                x11:=UOp
                                                x12:=UOp(
                                                    x13:=UOp(Op
                                                        x12,)),))
                                UOp(Ops.MUL, ar
                                    UOp(Ops.EXP2,
                                        UOp(Ops.MUL,
```

Fuse it to one kernel

Schedule 1 Kernel n1

▼ schedule to linear - 1 (2)

► prepare_rangeify (3)

▼ get_kernel_graph (9)

► run_rangeify (3)

symbolic+reduce_col

limit buffers

View Rangeify

stage to store - 3

split kernels

remove indexing from

remove views from th

View Kernel Graph

► resolve linear call - 1 (1)

create COPY kernels for 5

softmax

HCQ Compile 1 Kernel n1

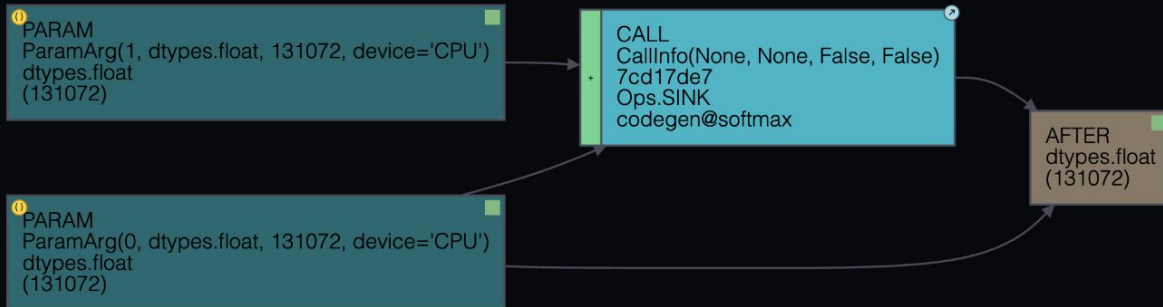
HCQ Link 1 Kernel n1

HCQ Compile 1 Kernel n2

HCQ Link 1 Kernel n2

HCQ Compile 1 Kernel n3

HCQ Link 1 Kernel n3



[rangeify.py:398](#)

```
if VIZ:  
graph_rewrite(tsink,  
PatternMatcher([]),  
name="View Kernel Graph")
```

```
UOp(Ops.SINK, arg=None, src  
UOp(Ops.AFTER, arg=None,  
x1:=UOp(Ops.PARAM, arg=  
UOp(Ops.CALL, arg=CallI  
UOp(Ops.SINK, arg=Ker  
UOp(Ops.END, arg=No  
UOp(Ops.STORE, ar  
UOp(Ops.INDEX,  
UOp(Ops.RESHA  
UOp(Ops.PAR  
x9:=UOp(Ops  
x10:=UOp(  
x11:=UOp(  
x12:=UOp(Ops.  
x10,)),  
x13:=UOp(Ops.  
x11,)),)),  
UOp(Ops.MUL, ar  
UOp(Ops.EXP2,  
UOp(Ops.MUL  
UOp(Ops.A  
UOp(Ops  
x19:=  
UOp  
x0
```

UOp kernels give full control over fusion

```
def solve(x):  
- return x.softmax(-1)  
+ return Tensor.custom_kernel(Tensor.empty_like(x), x, fxn=softmax)[0]
```



softmax

View Base AST

multi_pm

► early movement ops - 4

► load collapse (1)

split ranges

initial symbolic

simplify ranges

shift (3,) 4 upcast - 1

shift (2,) 4 unroll - 1

flatten range - 2

postopt symbolic - 2

expander - 4

remove reduces - 3

add local buffers

add gpudims

*** expand broadcast / ad

► devectorize2 - 162 (9)

early symbolic - 8

memory coalescing - 13

add images - 6

extra symbolic - 10

lower all index dtypes - 35

final symbolic

cast float alu operands

early decompositions - 1

decomp dtypes

late decompositions - 91

move gates from index

final rewrite - 43

cast consts - 147

add implicit barriers



- Show indexing (r)
- Show all CALL src (c)
- Show SINK (s)

[init .py:297](#)

```
if VIZ:  
graph_rewrite(ast,  
PatternMatcher([]),  
name="View Base AST")
```

```
<module> workshop.py:78  
if __name__ == "__main__":  
main workshop.py:62  
if args.command == "check":  
local_check workshop.py:32  
result = solve(*inputs).rea  
_wrapper tensor.py:769  
ret = fn(*args, **kwargs)  
inner contextlib.py:81  
return func(*args, **kwargs)  
realize tensor.py:415
```

```
c0 = UOp(Ops.PARAM, (), Par  
c3 = UOp(Ops.STACK, (UOp.co  
c5 = UOp.range(32, 0, AxisT  
c6 = UOp.range(4096, 3, Axi  
c8 = UOp(Ops.PARAM, (), Par  
c9 = UOp(Ops.RESHAPE, (c8,  
c11 = UOp.range(4096, 1, Ax  
c12 = c9.index(c5, c11)  
c15 = c12.reduce(c11, arg=(  
c20 = UOp.range(4096, 2, Ax  
c24 = ((c9.index(c5, c20)+c  
c27 = ((c9.index(c5, c6)+c1  
c29 = UOp(Ops.RESHAPE, (c0,  
ast = c29.sink(arg=KernelIn
```

tinygrad translates high level Python to C

```
def softmax(out, x):
    row = UOp.range(x.shape[0], 0)
    col_max = UOp.range(x.shape[1], 1, AxisType.REDUCE)
    row_max = x[row, col_max].reduce(col_max, arg=Ops.MAX)
    col_sum = UOp.range(x.shape[1], 2, AxisType.REDUCE)
    denominator = (x[row, col_sum] - row_max).exp().reduce(col_sum, arg=Ops.ADD)
    col = UOp.range(x.shape[1], 3)
    values = (x[row, col] - row_max).exp() / denominator
    store = out[row, col].store(values).end(col, row)
    return store.sink(arg=KernelInfo(name="softmax"))
```

```
typedef float float4 __attribute__((aligned(16),ext_vector_type(4)));
void softmax(float* restrict data0_131072, float* restrict data1_131072) {
    float buf0[1];
    float buf1[1];
    for (int Lidx0 = 0; Lidx0 < 32; Lidx0++) {
        int alu0 = (Lidx0<<12);
        *(buf0+0) = ((float)(-__builtin_inff()));
        for (int Ridx1 = 0; Ridx1 < 4096; Ridx1++) {
            float val0 = (*(data1_131072+(alu0+Ridx1)));
            float val1 = *(buf0+0);
            float alu2 = ((val1<val0)?val0:val1);
            *(buf0+0) = alu2;
        }
        float val2 = *(buf0+0);
        *(buf1+0) = 0.0f;
        for (int Ridx2 = 0; Ridx2 < 1024; Ridx2++) {
            float4 val3 = (*((float4*)((data1_131072+(alu0+(Ridx2<<2))))));
            float alu6 = ((val3[0]-val2)*1.4426950216293335f);
            _Bool alu7 = (alu6!=alu6);
            float alu8 = ((alu6!=((float)(-__builtin_inff())))?alu6:0.0f);
            float alu9 = (alu7?0.0f:alu8);
            float alu10 = ((alu6!=((float)(-__builtin_inff())))?alu9:0.0f);
            float alu11 = ((alu10<0.0f)?-0.5f:0.5f);
            int cast0 = ((int)((alu10+alu11)));
            float alu12 = ((val3[1]-val2)*1.4426950216293335f);
            _Bool alu13 = (alu12!=alu12);
            float alu14 = ((alu12!=((float)(-__builtin_inff())))?alu12:0.0f);
            float alu15 = (alu13?0.0f:alu14);
```

Challenge 3: hardware specific kernels

3.py > ...

```
1  # Program challenge: optimize signed int8 row-wise dot products using a custom CPU program.
2  # Inputs: int8 [256, 4096]. Output: int32 [256]. Use instructions supported by your evaluation CPU.
3  from tinygrad import dtypes
4
5  def solve(x, w):
6      return (x.cast(dtypes.int32) * w.cast(dtypes.int32)).sum(-1)
```

VIZ=1 python workshop.py check 3.py

Challenge 3: hardware specific kernels

This is a dot product using generic operations



r_256_1024_4

View Base AST

multi_pm

early movement ops

► load collapse (1)

split ranges

initial symbolic

simplify ranges

shift (0,) 4 unroll - 1

flatten range - 1

postopt symbolic - 1

expander - 2

remove reduces - 3

add local buffers

add gpudims

*** expand broadcast / add lc

devectorize2 - 25

early symbolic - 3

memory coalescing - 9

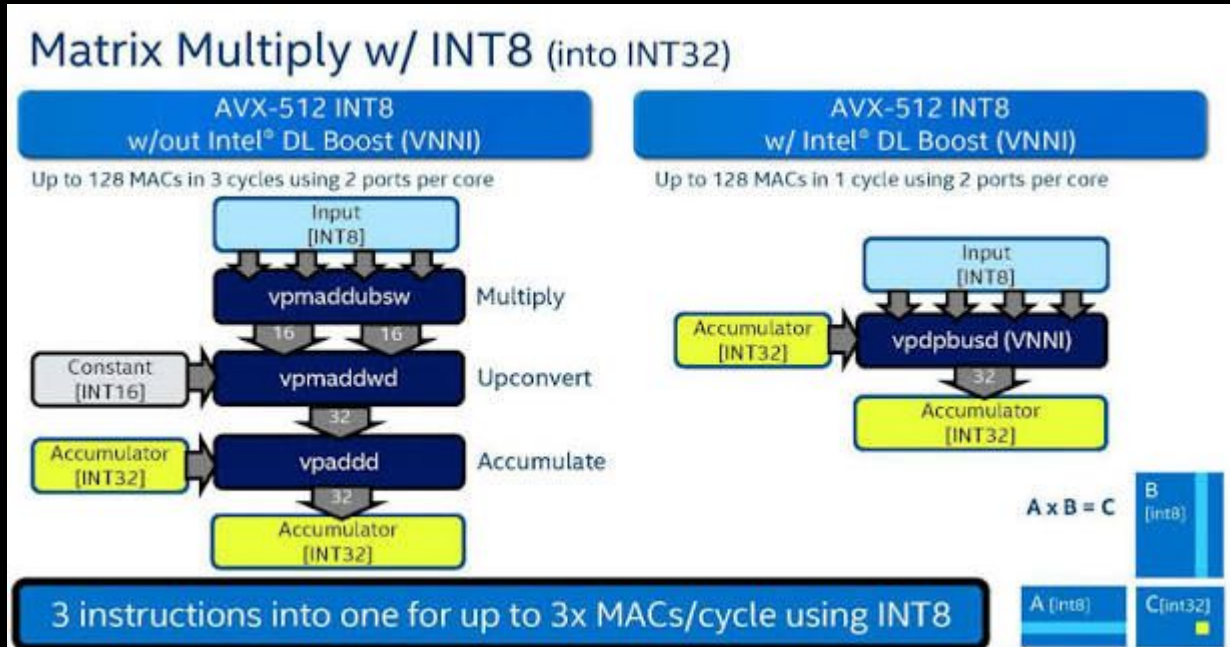
add images - 7

extra symbolic - 1

lower all index dtypes - 24

```
void r_256_1024_4(int* restrict data0_256, signed char* restrict data1_1048576, signed char* restrict data2_1048576)
{
    int buf0[1];
    for (int Lidx1 = 0; Lidx1 < 256; Lidx1++) {
        *(buf0+0) = 0;
        for (int Ridx0 = 0; Ridx0 < 1024; Ridx0++) {
            int alu1 = ((Ridx0<<2)+(Lidx1<<12));
            int alu2 = (alu1+1);
            signed char val0 = (*(data1_1048576+alu2));
            int alu3 = (alu1+2);
            signed char val1 = (*(data1_1048576+alu3));
            int alu4 = (alu1+3);
            signed char val2 = (*(data1_1048576+alu4));
            signed char val3 = (*(data1_1048576+alu1));
            signed char val4 = (*(data2_1048576+alu2));
            signed char val5 = (*(data2_1048576+alu3));
            signed char val6 = (*(data2_1048576+alu4));
            signed char val7 = (*(data2_1048576+alu1));
            *(buf0+0) = (*(buf0+0))+(((int)(val3))*((int)(val7)))+(((int)(val0))*((int)(val4)))+(((int)(val1))*((int)(val5)))+(((int)(val2))*((int)(val6)));
        }
        *(data0_256+Lidx1) = *(buf0+0);
    }
}
```

Hardware has special dot product instructions



Agents can read manuals and write hardware specific code

< > ↺

amd.com/content/dam/amd/en/documents/instinct-tech-c

☰ CDNA4 Instruction Set Architecture: Reference Guide 1 / 608

x86 and amd64 instruction reference

Derived from the December 2023 version of the [Intel® 64 and IA-32 Architectures Software Developer's Manual](#). Last updated 2024-02-18.

THIS REFERENCE IS NOT PERFECT. It's been mechanically separated into distinct files by a dumb script. It may be enough to replace the official documentation where money is at stake, go get the official and freely available documentation.

Core Instructions

Mnemonic	Summary
AAA	ASCII Adjust After Addition
AAD	ASCII Adjust AX Before Division
AAM	ASCII Adjust AX After Multiply
AAS	ASCII Adjust AL After Subtraction
ADC	Add With Carry

Agents can read manuals and write hardware specific code

```
void r_256_1024_4(int* restrict data0_256, signed char* restrict data1_1048576, signed char* restrict
int buf0[1];
for (int Lidx1 = 0; Lidx1 < 256; Lidx1++) {
    *(buf0+0) = 0;
    for (int Ridx0 = 0; Ridx0 < 1024; Ridx0++) {
        int alu1 = ((Ridx0<<2)+(Lidx1<<12));
        int alu2 = (alu1+1);
        signed char val0 = (*(data1_1048576+alu2));
        int alu3 = (alu1+2);
        signed char val1 = (*(data1_1048576+alu3));
        int alu4 = (alu1+3);
        signed char val2 = (*(data1_1048576+alu4));
        signed char val3 = (*(data1_1048576+alu1));
        signed char val4 = (*(data2_1048576+alu2));
        signed char val5 = (*(data2_1048576+alu3));
        signed char val6 = (*(data2_1048576+alu4));
        signed char val7 = (*(data2_1048576+alu1));
        *(buf0+0) = (((*(buf0+0))+(((int)(val3))*(((int)(val7)))+(int)(val0))*(((int)(val4)))+(int)(val1)))+(int)(val2));
    }
    *(data0_256+Lidx1) = (*(buf0+0));
}
```



```
#include <emmintrin.h>
void dot_i8(int * restrict out, const signed char * restrict x, const signed char * restrict w) {
    for (int row = 0; row < 256; row++) {
        __m128i sum = _mm_setzero_si128(), zero = _mm_setzero_si128();
        for (int col = 0; col < 4096; col += 16) {
            __m128i a = _mm_loadu_si128((const __m128i *) (x + row * 4096 + col));
            __m128i b = _mm_loadu_si128((const __m128i *) (w + row * 4096 + col));
            // Sign-extend int8 to int16, multiply pairs and accumulate into int32.
            __m128i sa = _mm_cmpgt_epi8(zero, a), sb = _mm_cmpgt_epi8(zero, b);
            sum = _mm_add_epi32(sum, _mm_madd_epi16(_mm_unpacklo_epi8(a, sa), _mm_unpacklo_epi8(b, sb)));
            sum = _mm_add_epi32(sum, _mm_madd_epi16(_mm_unpackhi_epi8(a, sa), _mm_unpackhi_epi8(b, sb)));
        }
        sum = _mm_add_epi32(sum, _mm_srli_si128(sum, 8));
        sum = _mm_add_epi32(sum, _mm_srli_si128(sum, 4));
        out[row] = _mm_cvtsi128_si32(sum);
    }
}
```

Agents can optimize AI kernels

<IMI

Products

Features

Research

Resources

Pricing

Help

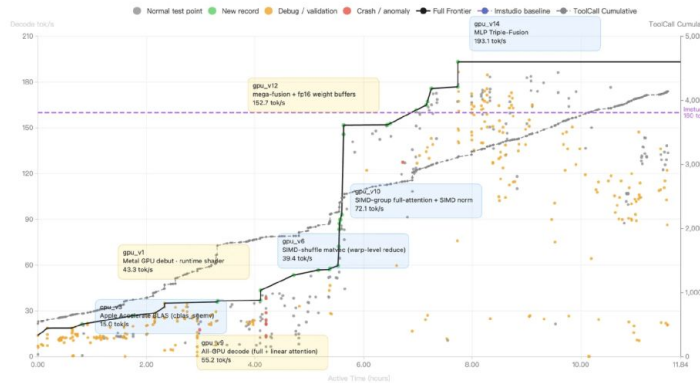
Models

For

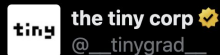
Mac. By implementing and optimizing model inference in **Zig**—a highly niche programming language—it demonstrated exceptional out-of-distribution generalization. Across **4,000+ tool calls**, over **12 hours of continuous execution**, and **14 iterations**, Kimi K2.6 dramatically improved throughput from ~15 to ~193 tokens/sec, ultimately achieving speeds ~20% faster than LM Studio.

LLM Inference on M3 Max — Performance Evolution

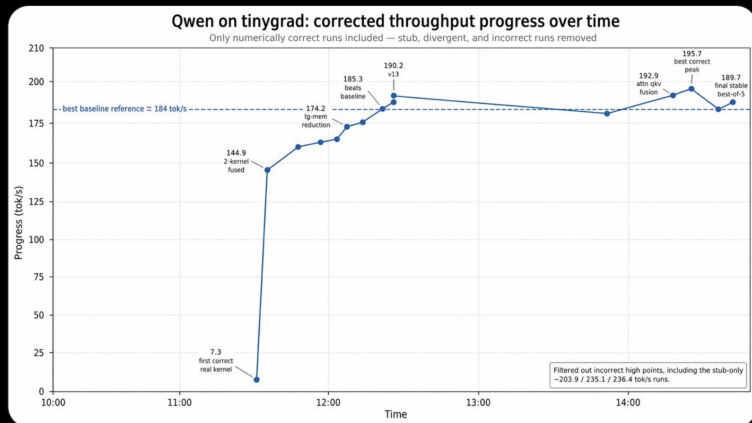
X-axis = Active Time (hours) · Y-axis = Decode tok/s



tinygrad is agent friendly



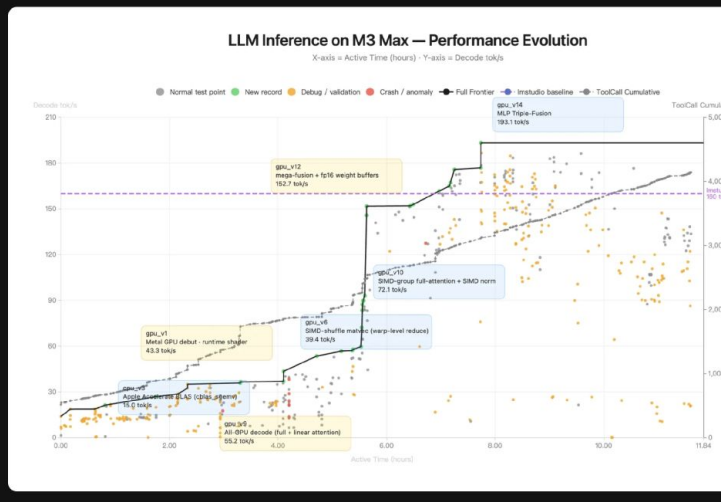
We set out to replicate Kimi's 193 tok/s Qwen3.5-0.8B on M3 Max. Our baseline is already 178 tok/s, beating LMStudio (160) and llama.cpp (140) out of the box, but with tinygrad's custom kernel feature Claude cranked it to 195.7!



<IMI

Products Features Research Resources Pricing Help Models

Mac. By implementing and optimizing model inference in **Zig**—a highly niche programming language—it demonstrated exceptional out-of-distribution generalization. Across **4,000+ tool calls**, over **12 hours of continuous execution**, and **14 iterations**, Kimi K2.6 dramatically improved throughput from ~15 to ~193 tokens/sec, ultimately achieving speeds ~20% faster than LM Studio.



We also sell AI computers

chestnut



ready to drive

Everything you need to use chestnut with comma four.

SHIPS IN 4-12 WEEKS

\$799



tiny chestnut eGPU dock

Use chestnut with your own GPU and power supply.

IN STOCK

\$249

ADD TO CART (SHIPS IN 4-12 WEEKS)



We also sell AI computers

chestnut



ready to drive

Everything you need to use chestnut with comma four.

SHIPS IN 4-12 WEEKS

\$799



tiny chestnut eGPU dock

Use chestnut with your own GPU and power supply.

IN STOCK

\$249

ADD TO CART (SHIPS IN 4-12 WEEKS)



Select a tinybox



tinybox green v1
\$18,888.88 USD



tinybox green v2
\$38,888.88 USD



tinybox green v2 with 4x RTX Pro tinybox red v2 8000 Blackwells
From \$31,888.88 USD



tinybox red v2
\$12,888.88 USD



tinybox pro v2
From \$188,888.88 USD



the tiny corp

@_tinygrad_

245 single user tok/s on DeepSeek-V4-Flash-0731, and it's only using 2 of the RTX 6000 Blackwell GPUs! In honor of DeepSeek, we're launching a 2 GPU edition of our tinybox. All hardware to install 4 GPUs is included and tested.

Benchmark results - DeepSeek-V4-Flash-0731 @ tinybox

Answer first: it's running on 2 GPUs (TP=2 on physical GPU 0 + GPU 1, ~95 GB used each; GPUs 2 and 3 idle).

Scenario	Output tok/s	Notes
Single user (CC1, 128+512)	~245 tok/s sustained	median TPOT ~2.8 ms + ~380-350 tok/s instantaneous; TTFT ~0.4 s
Concurrent *16 (128+512)	688 tok/s total	TPOT ~17 ms per stream; D5park accept 63.8%
Concurrent *16, long ctx (6192+512)	499 tok/s decode	+8,477 tok/s total incl. prefill (~10.6k prefill tok/s); D5park accept 80.0%, accept-len 5.92

Settings: vLLM "Gilded Gnosis" z1g, 012X W6A8 kernels, D5park HS fixed-depth speculative decode, fp8 HW cache, 131k max context, temp 0, random-token prompts (long-ctx acceptance is inflated by repetitive tokens - real code ~ 64k).

For reference: 245 tok/s single-user actually beats the runbook's 217-220 tok/s validated number. Concurrency is capped at 16 by the release profile (MAX_NUM_SEQS=16).

working at tinygrad office in HK (Sheung Wan)



the tiny corp 🌟 @__tinygrad__ · Apr 15



Article

tinygrad is hiring

working at tinygrad office in HK (Sheung Wan)



the tiny corp 🌟 @__tinygrad__ · Apr 15



tinygrad is hiring

A1 Short Description					
	A	B	C	D	E
	Short Description	Type	Value	Hardware Required	Link
1					
2	3090 or 4090 or 5090 works over USB 3.0 on chestnut with our USB stack	Feature	\$500	Chestnut + NVIDIA GPU	
3	Mergable USB4 support for chestnut	Feature	\$600	Chestnut + any GPU	
4	QCOM A630 emulator	Feature	\$500	None!	
5	Cycle accurate RDNA3 emulator (add SQTT support to emulator matching real hw on SRAM only kernels)	Feature	\$1,000	RDNA3 GPU	
6	Buffers and args must be supported in any input order on all backends, with tests and taste. No AI for this.	Refactor	\$300	None!	
7	GPUccelot or similar that works with CUDA 12 and replaces our CI NV tests with this	Upgrade	\$500	None!	

working at tinygrad office in HK (Sheung Wan)

discord:



the tiny corp 🟡 @__tinygrad__ · Apr 15



Article

tinygrad is hiring

Short Description					
	A	B	C	D	E
	Short Description	Type	Value	Hardware Required	Link
1					
2	3090 or 4090 or 5090 works over USB 3.0 on chestnut with our USB stack	Feature	\$500	Chestnut + NVIDIA GPU	
3	Mergable USB4 support for chestnut	Feature	\$600	Chestnut + any GPU	
4	QCOM A630 emulator	Feature	\$500	None!	
5	Cycle accurate RDNA3 emulator (add SQTT support to emulator matching real hw on SRAM only kernels)	Feature	\$1,000	RDNA3 GPU	
6	Buffers and args must be supported in any input order on all backends, with tests and taste. No AI for this.	Refactor	\$300	None!	
7	GPUocelot or similar that works with CUDA 12 and replaces our CI NV tests with this	Upgrade	\$500	None!	